

土木工程規範問答系統（RAG 雙軌對照版）

以檢索增強生成（RAG）為核心的土木規範 PDF 智慧問答系統

GitHub 專案連結：<https://github.com/ching-pochiu/rag>

一、專題動機

土木工程規範文件（CNS 標準、施工規範等）動輒數百頁，內含大量表格化數據（鋼筋牌號、材料容許差、強度規定等），工程人員在審查或施工時常需要「翻頁找數字」，效率低且容易漏看。本專題以 RAG 架構讓使用者用自然語言提問即可快速定位正確條文與數值，並透過多層機制降低 LLM 生成答案時「編造數字」的風險——這對規範查詢這種容錯率極低的場景尤其重要。

與原型 ConSpec-Agent 的差異

本專題最初源自筆者在團隊專題 ConSpec-Agent（工程品質審查系統）中負責的法規問答模組。實際比較後，發現原模組的風險在於「檢索到什麼就直接信什麼」——沒有排序品質驗證，也沒有生成後的事實查核，對規範查詢這種容錯率極低的場景並不安全。因此把這部分獨立拉出來重新設計：

	ConSpec-Agent 原模組	這個獨立專題
檢索方式	單純向量檢索	向量 + BM25 + RRF 融合排序
排序品質把關	無（向量分數直接排序）	Cross-Encoder Reranker 二次精選
生成後查核	無	Groundedness 檢查 + 引用頁碼反查
表格資料處理	原始文字直接餵給 LLM	結構化清理成乾淨表格，並依牌號/代號分組
關鍵數值查詢	完全依賴 LLM 生成	額外提供規則式查表，雙軌互相驗證

也就是說，這不是「重做一個類似的東西」，而是拿同一個問題域，把原本「檢索到就相信」的單軌設計，換成「檢索、排序、生成、查核」四層都各自有把關機制的架構，並用系統性測試逐一驗證每一層真的有解決問題。

二、系統架構

系統分為「索引建立」「檢索與生成」「生成後驗證」三個階段：

索引建立：上傳 PDF → pdfplumber 解析（頁碼還原、表格結構化清理）→ 表格依牌號/代號分組逐塊切割、條文依節號/子項邊界切割 → HuggingFace Embedding 分批寫入向量庫（Chroma）+ BM25 關鍵字索引（中文逐字元分詞）+ SD/SR/D 規則式代號索引

檢索與生成：使用者問題同時做向量搜尋與 BM25 搜尋 → RRF 融合排序（向量核可閘門 + BM25 前 3 名一律放行）→ Cross-Encoder Reranker 精選 → 組成 Context → 交給 Gemini 生成回答

生成後驗證：Groundedness 檢查（答案數字是否有原文依據）+ 引用頁碼反查（頁碼是否真的被檢索到），並列出 SD/SR/D 規則式查表結果做雙軌對照

三、技術亮點

一般入門 RAG 教學通常只有「向量檢索 → LLM 生成」兩步，本專題額外做了以下強化：

環節	做法	目的
PDF 頁碼還原	逐頁偵測印刷頁碼，找不到的用鄰近錨點局部內插	附錄重新編號、章節跳頁時，引用頁碼仍正確
表格結構化清理	合併多列表頭、向下補值、去重複列，多重特徵辨識資料列	把 PDF 表格斷裂、跨頁的原始資料還原成乾淨表格
表格依代號分組切塊	依符號/牌號重複分組或純代號格式逐列切開，摘要句明講是哪個代號	避免大表格被當一個 chunk，稀釋單一代號查詢的相關性
中文 BM25 逐字分詞	英數字連續當一個 token、中文逐字拆開的正則分詞	修正中文無空白導致 BM25 全部零分、退化成雜訊的問題
混合檢索 + RRF	向量語意搜尋 + BM25 關鍵字搜尋，依排名加權融合	語意與精確關鍵字互補，比簡單聯集更可靠
向量核可閘門動態放寬	向量前 k 名 ∪ BM25 前 3 名才放行，不限代號類查詢	修正一般問句被閘門誤擋、答成查無相關規定的問題
Cross-Encoder Reranker	用 BAAI/bge-reranker-base 對候選重新精算相關性	比向量/BM25 各自獨立打分更準，是最後一道把關
生成 Prompt 措辭校準	避免過度強調「數值」，規定/程序皆可直接回答	修正程序性問題因用詞被誤判無數值可答而拒答
Groundedness 檢查	比對答案數值是否出現在原文中，濾除列表編號誤報	攔截「檢索沒命中、LLM 卻自己編數字」的幻覺
引用頁碼反查	答案引用頁碼若不在檢索範圍內，主動跳警告	防止 LLM 編造看似合理但查無來源的引用
規則式雙軌對照	正則規則直接從表格索引查 SD/SR/D 代號數據	不透過 LLM，提供獨立、可驗證的查詢路徑
索引建立分批 Embedding	冷啟動建索引時分批寫入向量庫	降低尖峰記憶體用量，避免資源有限環境下建索引被砍掉

四、成效驗證（量化比較）

不只是文字描述「有變好」，以下是實測過的具體案例：

Groundedness 誤報率：修正前後對照

	修正前	修正後
測試輸入	含列表編號的答案（如「1. 降伏強度 420MPa」「2. 抗拉強度 620MPa」「(3) 伸長率 12%」）	同一段答案
誤報數量	3 個（1、2、3 被誤判為查無原文依據）	0 個
原因	直接抓答案裡所有數字比對	比對前先濾除行首列表編號，只比對真正的數值

BM25 中文分詞：修正前後對照

實測查詢「SD420W和SD690的抗拉強度和降伏強度」（句子沒有任何空白）：

	修正前	修正後
分詞結果	整句被當成唯一一個 token，跟語料庫任何文件都對不上	拆成「SD420W」「和」「SD690」等有意義的 token
BM25 前 2 名	跟問題完全無關的文件內容（分數接近雜訊）	SD420W、SD690 專屬表格 chunk（分數 27.7、27.2）

生成 Prompt 措辭：修正前後對照

實測查詢「鋼筋重驗不合格時怎麼處理」：

	修正前	修正後
檢索狀態	正確段落已檢索到，Reranker 分數 0.965（滿分等級）	同一份 context
最終答案	「查無相關規定，請人工確認」	正確列出 7.4 節重驗程序
原因	Prompt 反覆強調「數值」，模型誤判程序性問題為無數值可答	措辭改為數值、規定或處理程序皆可直接回答

表格辨識覆蓋率：全面掃描前後對照

系統性掃描兩份規範全部頁面，比對「pdfplumber 有偵測到表格結構」與「清理後實際產出乾淨表格」的差異：

	修正前	修正後
疑似辨識失敗頁面數	24 頁	11 頁
剩餘頁面性質	部分為真正表格辨識失敗	逐頁人工確認，全部為封面／目錄／修訂歷史等本來就不是表格的內容

測試規模：以 20 題 → 50 題 → 52 題 → 57 題逐步擴大的測試集反覆驗證每一輪修正，每次新增修正都重新跑過先前已通過的題目（包含刻意重測「向量閘門放寬後，原本靠代號跳過閘門的查詢有沒有跟著變差」這類回歸案例），確認舊修正沒有被新修正打壞，才視為修正生效。

五、實際運作畫面

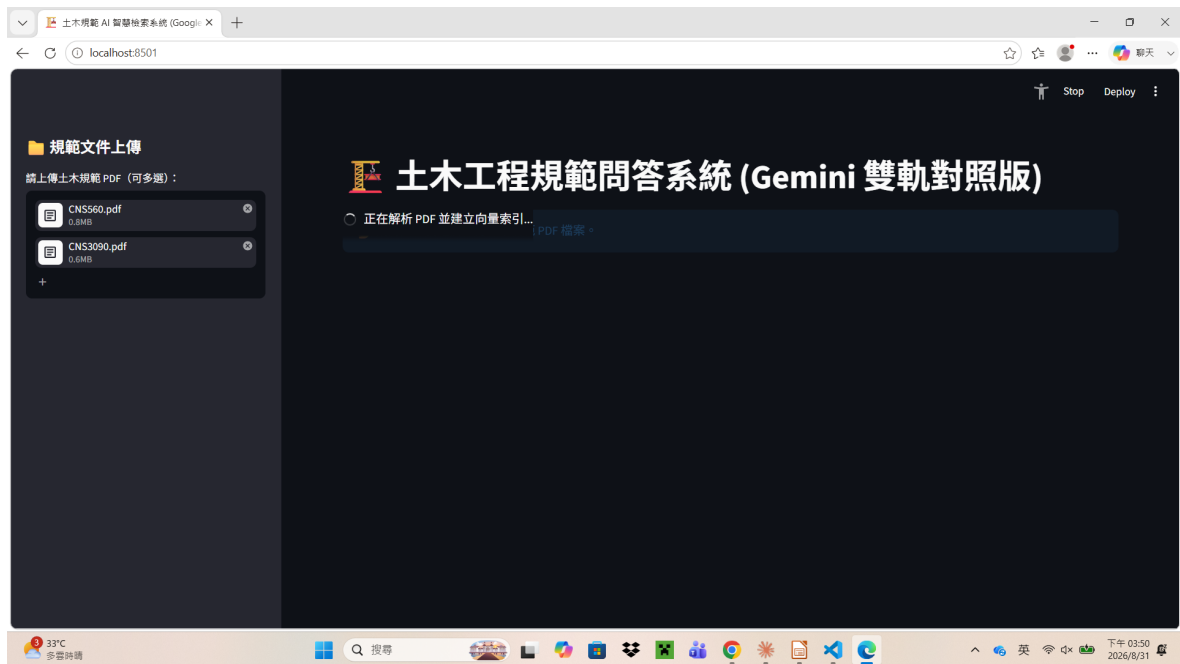


圖 1：上傳規範 PDF、建立索引



圖 2：AI 智慧摘要——精確引用頁碼與表格編號回答

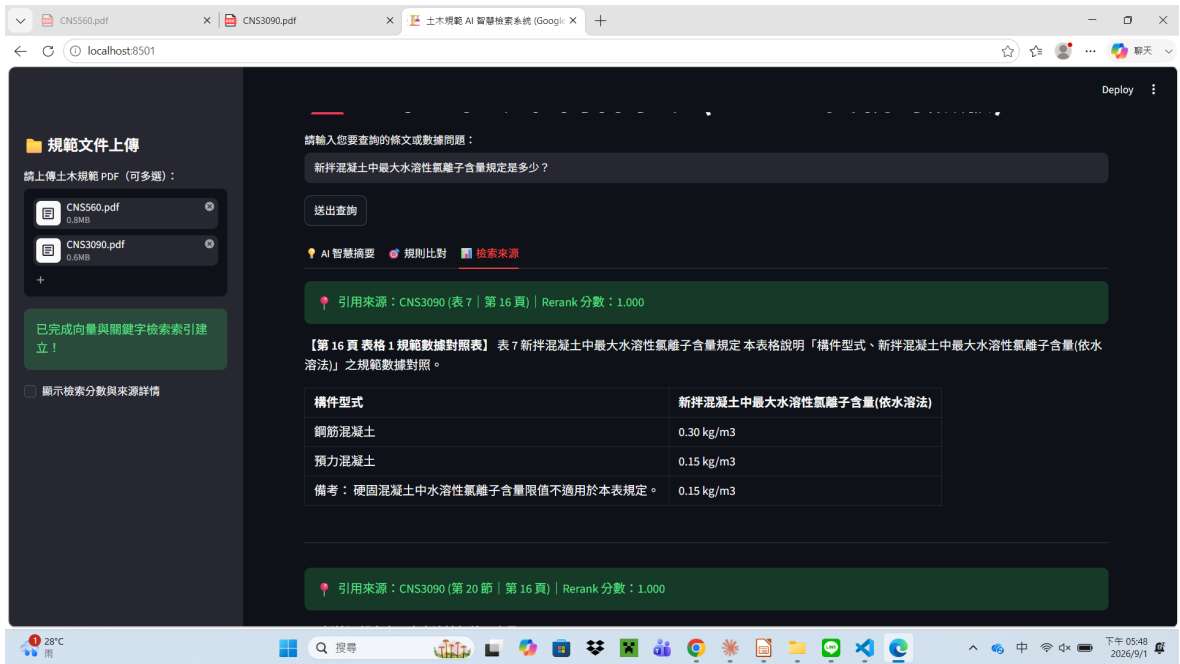


圖 3：檢索來源——可看到每筆候選的 Reranker 分數



圖 4：規則比對——SD/SR 鋼筋牌號直接查表，與 AI 答案雙軌對照

六、技術棧

類別	選用技術
前端／應用框架	Streamlit
PDF 解析	pdfplumber
向量資料庫	Chroma
Embedding 模型	sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
Reranker 模型	BAAI/bge-reranker-base

類別	選用技術
關鍵字檢索	BM25 (rank_bm25)
LLM	Google Gemini (透過 langchain-google-genai)
框架整合	LangChain

七、已知限制與未來規劃

模型選擇受硬體限制：Reranker 原採用效果更好的 BAAI/bge-reranker-v2-m3 (5.68 億參數)，但在記憶體有限的機器上會導致程序被系統強制終止，因此改用較輕量的 bge-reranker-base，是準確度與可運行性之間的取捨。

Groundedness 檢查仍屬粗略比對：目前以數字集合差集判斷，未涵蓋單位、語意層級的一致性檢查。

語料範圍：目前僅支援使用者臨時上傳的 PDF，尚無固定的規範資料庫版本管理機制。

規則式查表覆蓋範圍：目前支援 SD/SR 鋼筋牌號與 D 標稱尺寸代號兩種格式，其他規範數值（如混凝土強度等級、坍度數值區間）需要「數值範圍比對」而非「精確代號比對」，屬於不同複雜度的擴充，尚未涵蓋。

窮舉型清單可能列不全：問「列出所有 XX」這種需要彙整多個 chunk 才能答全的問題，受限於送進 LLM 的候選數上限，實測會出現只答出部分項目的情況，是候選數量與答案精簡度之間的取捨。

短句 chunk 對通用字查詢的命中率較低：中文 BM25 採逐字元分詞，若條文本身很短且查詢只共用單一常見字（而非多字詞組），字元層級的區分度不夠，可能檢索不到——這是逐字元分詞方法的結構性限制。

LLM 不會主動彙整分散片段：若答案需要的資訊分散在多個獨立段落，即使每個片段都有被正確檢索到、分數也過門檻，LLM 傾向保守回答查無相關規定而非主動整理成清單，這是生成端而非檢索端的限制。

完整原始碼、README 與更多截圖請見 GitHub：<https://github.com/ching-pochiu/rag>